Anumati: Proof of Adherence as a Formal Consent Model for Autonomous Agent Protocols

Ravi Kiran Kadaboina

Independent Researcher

M.S., Computer Engineering, University of New Mexico, 2011

Anumati is Sanskrit for "consent" or "formal permission."

Abstract

As autonomous AI agents increasingly call other agents to complete tasks on behalf of a human principal, a structural accountability gap has emerged: the calling agent accepts the terms of service of the callee without any protocol-level mechanism to prove that it understood those terms or that it subsequently honoured them. Authentication protocols such as OAuth and mutual TLS establish *who* may call *which* capability. They do not address *under what conditions* a permitted call may be made, and those conditions change as the callee's policies evolve. In this paper we formalise the distinction between *proof of acceptance* (a timestamped acknowledgement) and *proof of adherence* (a per-action reasoning record citing the specific clause evaluated). We propose three primitives (PolicyDocument, ConsentRecord, and AdherenceEvent) that together constitute a versioned, append-only consent model for agent-to-agent communication. The model is instantiated as a non-breaking extension to two widely used agent protocols: the Agent2Agent (A2A) protocol and the Model Context Protocol (MCP). A TLA+ specification of the consent lifecycle, together with a reference Python implementation of the chain integrity and adherence trail validators, is available in the accompanying repository.

---

1. Introduction

In this section we discuss the motivation leading to our work, an overview of the current state of consent handling in agent protocols, and our approach to closing the accountability gap that we identify. Section 1.1 gives background on how calling agents today commit their principals to the terms of the services they call. Section 1.2 describes the motivation upon which the work was developed, including the empirical evidence that agents are not honouring even basic exclusion signals today, and the regulatory pressure under the EU AI Act. Section 1.3 describes related work in single-agent governance, runtime enforcement, and privacy standards. Section 1.4 describes the specific contributions of this work and our approach.

1.1 Background

The Agent2Agent protocol (A2A) [4] standardises how one agent advertises its capabilities and authenticates another. The Model Context Protocol (MCP) [5] plays an analogous role for communication between an orchestrator agent and the tools or data sources it consumes. Both protocols delegate security to standard HTTP authentication mechanisms such as OAuth 2.x, API keys, OpenID Connect, and mutual TLS. These mechanisms are well designed for the question they answer: given a request, is the requesting agent permitted to invoke the requested capability.

A second category of rules governs something different. Consider an agent that provides a data analysis service. Authentication tells the calling agent whether it is permitted to invoke the `analyse_dataset` skill. It says nothing about whether the calling agent may store the task output beyond the current session, whether it may aggregate results across multiple principals, whether it is obligated to notify its human principal before invoking skills above a given cost threshold, or whether it may share results with a third agent the callee has not authorised. These constraints are contextual, compositional, and they change as the callee's policies evolve. They cannot be encoded in OAuth scopes, which are discrete, static, and binary. Scopes answer "permitted or not." Usage policy answers "permitted, under these conditions, as of this version, subject to change."

Further complicating the picture, under the Uniform Electronic Transactions Act (UETA) §14, contracts formed by electronic agents bind the human principal, even without their awareness [7, 8]. As callee policies evolve and calling agents continue operating under stale terms, the principal's legal exposure compounds silently.

1.2 Motivation

The gap is not theoretical. The 2025 AI Agent Index surveyed 30 deployed agents and found that only 6 of them explicitly stated that their crawler bots respect robots.txt, while 16 provided no clear statement about web exclusion compliance at all [1]. Robots.txt is a proxy: if agents ignore the most basic signal of "do not access this," there is no reason to expect that they will honour more nuanced usage terms. The problem is empirical and is worsening as agent deployment scales.

The EU AI Act (Regulation (EU) 2024/1689) compounds the urgency. The Act enforces in phases. Prohibited AI practices took effect in February 2025 and general-purpose AI model obligations from August 2025. The obligations most relevant to autonomous agent transactions, namely Art. 14 (human oversight of high-risk AI systems) and Art. 50 (transparency obligations including disclosure of AI-generated content and automated decision-making), take effect on August 2, 2026 for high-risk systems listed in Annex I and Annex III [2]. As of this writing, the EU AI Office has published no technical guidance addressing consent mechanisms for autonomous agent-to-agent transactions [3]. For systems in

scope, effective oversight requires, at minimum, that the principal can determine what policies the agent agreed to, what clauses it evaluated at each action, and what reasoning it applied. The present model is designed to provide exactly that audit surface in advance of such guidance, so that implementations are not built retroactively under deadline pressure.

Several years ago I implemented a versioned terms-of-service system at a consumer technology company that tried to take this problem seriously. Policy documents were managed in a headless content management system (CMS) with full version history, each published version immutable and content-addressed. The authentication state machine, built on a custom OAuth 2-based flow, checked on every session whether the user's most recently accepted document version matched the current published version. If not, the session was blocked and the user was presented with the new terms before proceeding. Every acceptance event was stored with a pointer to the exact document version and a reference to the prior acceptance record, forming a singly-linked chain per user. The goal was legal defensibility: in any dispute, the system could show precisely which version of the terms a user had accepted, and when, across the full history of their account.

The data was unsurprising. Median time on the acceptance screen was under four seconds. The linked chain proved acceptance with precision; it proved nothing about comprehension or subsequent compliance. That experience motivates the model we propose here. Autonomous agents, unlike humans, are capable of parsing a policy clause by clause, evaluating each at every action, and leaving a reasoning trail that a regulator or principal can inspect after the fact. The consent infrastructure available to them today does not ask them to do any of this.

1.3 Related Work

A family of recent proposals addresses agent governance from adjacent directions. The Open Agent Governance Specification (OAGS) [12] defines five governance primitives but is explicitly "local-first"; it governs what a single agent may do in its own environment rather than what two agents agree to when they interact. OpenMandate [15] takes a similar single-agent stance using declarative YAML mandates. Policy Cards [16] make an agent's constraints inspectable through machine-readable artefacts, and the `PolicyDocument` we introduce is the callee-side equivalent of a Policy Card.

At the runtime-enforcement layer, PCAS [17] compiles Datalog-derived policies into instrumented agents that are compliant by construction. MI9 [19] detects drift in agent behaviour after the fact. Both address a different layer than the bilateral consent model we propose; a deployment could use PCAS for local enforcement and the present model for cross-agent consent in parallel.

Governance-as-a-Service (GaaS) [18] proposes an external governance agent supervising other agents at runtime. The AIGA Internet-Draft [14] proposes

tiered risk-based governance covering action authorisation and audit logging. Both govern *what* an agent does; the model we propose governs *under what agreed terms*.

For financial services, the FINOS AI Governance Framework v2.0 [13] defines MI-21 (Agent Decision Audit and Explainability) with tiered audit logging up to cryptographic tamper-evidence. MI-21 is conceptually close to the adherence trail we describe in §3.3 but is explicitly silent on consent versioning.

On the privacy and consent standards side, IEEE P7012 [20] defines machine-readable personal privacy terms, and W3C DPV v2.2 [21] provides standardised vocabularies for data processing activities. The Kantara Consent Receipt [22] established early principles (consent as receipt, machine-readability, user portability) that inform the `ConsentRecord` design in §3.2.

1.4 Our Approach

We approach the problem by introducing three primitives that together form a versioned, append-only consent model between any two agents. `PolicyDocument` is the callee's machine-readable usage policy, content-addressed and semver-versioned. `ConsentRecord` is the calling agent's parsed understanding of that policy, with an entry for every clause. `AdherenceEvent` is a per-action record that cites the specific clause evaluated, the decision reached, and the natural-language reasoning the agent applied. The three primitives together produce two linked lists (a consent chain and an adherence trail) which together satisfy five properties (completeness, traceability, tamper evidence, version fidelity, and optional ledger anchoring) relevant to legal and regulatory accountability.

We instantiate the model as a non-breaking extension to A2A [4] and MCP [5] using each protocol's existing extension mechanism. We emphasise that the instantiations require no changes to the A2A or MCP core specifications; both protocols already provide native extension mechanisms that our protocol, the Agent Consent and Adherence Protocol (ACAP), uses without modification, and enforcement is carried out by middleware at the caller and callee boundaries rather than by the core runtime. We specify the consent lifecycle as a TLA+ state machine and verify seven safety properties and two liveness properties under the TLC model checker. A reference Python implementation of the canonical hashing, chain validator, and adherence trail validator is provided in the accompanying repository, together with 35 unit tests that exercise the structural invariants described in §3.

Section 2 discusses why authentication alone is insufficient for usage policy governance between agents. Section 3 introduces the three primitives and the formal verification of the consent lifecycle. Section 4 shows how the model integrates into A2A and MCP without modification of their core specifications. Section 5 surveys related work in more detail. Section 6 discusses the known limitations of the model, including the self-attestation boundary and agent ephemerality. Section 7 concludes.

The TLA+ specification, TLC configuration, protobuf schema, and reference Python implementation are available at: https://github.com/ravikiran438/agent-consent-protocol

---

2. The Consent Gap in Agent Protocols

2.1 What Authentication Covers (and What It Does Not)

A2A supports OAuth 2.x, API keys, OpenID Connect, and mutual TLS [4]. MCP uses OAuth 2.1 with incremental scope negotiation following its November 2025 specification update [6]. Both are well designed for their stated purpose: establishing *who* may invoke *which* capability.

Usage policy governs something different. As described in §1.1, a callee agent providing a data analysis service cannot express, through OAuth scopes alone, whether the calling agent may store task output beyond the current session, aggregate results across multiple principals, notify its human principal before expensive skill invocations, or share results with a third agent. These constraints are contextual, compositional, and evolve as the callee's policies change. They cannot be encoded in OAuth scopes, which are discrete, static, and binary.

2.2 The Version Problem

Human-facing terms of service change. Services update their policies, publish new versions, and expect users to re-accept. In practice this is handled by a login gate: the user sees a banner, clicks agree, and the session proceeds. The mechanism is crude but functional.

For agents, no equivalent mechanism exists in current protocols. Neither A2A nor MCP defines a versioned usage policy object, a typed consent record, or a mechanism to block skill invocation until a new policy has been processed [4, 5]. When a callee updates its terms, calling agents have no protocol-level notification and no structured way to record re-acceptance. Thus the liability accumulates silently, and the human principal identified through UETA §14 ends up bound to terms the agent has never in fact evaluated.

2.3 Proof of Acceptance vs. Proof of Adherence

We distinguish two consent properties.

Proof of acceptance is the traditional model: a timestamped record binding an identity to a document version at a point in time. It proves a party *agreed*. It proves nothing about whether the party *understood* the terms or *subsequently complied* with them.

Proof of adherence is what agents can uniquely provide: a per-action record citing the specific policy clause evaluated, the agent's reasoning, and the en-

forcement decision. It proves the agent *evaluated* the relevant clause *before acting*.

Table 1 summarises the distinction.

Dimension	Proof of Acceptance	Proof of Adherence
Granularity	Whole document	Per-clause, per-action
Timing	At acceptance event	At every action attempt
Content	Timestamp + identity	Clause citation + reasoning
Understanding	Assumed	Verified (parsed claims)
Post-acceptance compliance	Unverifiable	Auditable
Legal value	Proves agreement	Proves agreement *and* compliance

Agents are capable of proof of adherence because they can reason about policy text. The consent infrastructure in current agent protocols simply does not ask them to. Closing that gap is the work of this paper.

---

3. A Formal Consent Model

We define three primitives. Together they constitute a versioned, append-only consent model for agent-to-agent communication. We refer to the resulting protocol as the **Agent Consent and Adherence Protocol (ACAP)**.

3.1 PolicyDocument

A `PolicyDocument` is the machine-readable equivalent of a terms-of-service document. It is versioned using semantic versioning, content-addressed via SHA-256, and published by the callee agent at a well-known HTTPS URL.

Formally, a `PolicyDocument` P is a tuple:

```
P = (version, hash, effective_date, supersedes, claims,
publisher, natural_language_uri)
```

where `version` is a semver string, `hash` is the SHA-256 digest of the canonical JSON serialisation of P with the `hash` field set to the empty string (which breaks the circularity such that the digest input never includes itself), `effective_date` is an ISO 8601 timestamp, `supersedes` is the version of the document P replaces (if any), `claims` is a sequence of `PolicyClaim` objects (order preserving; claim order is significant for display and diff computation), `publisher` is the callee agent's identifier (DID or HTTPS URL), and `natural_language_uri` is the URL of the human-readable document from which the claims are derived.

A `PolicyClaim` c is a tuple:

```
c = (id, clause_ref, action, asset, rule_type, constraint,
since_version,
```

```
     category, dimension)
```

where `rule_type` is one of {permission, prohibition, obligation}, `action` and `asset` use ODRL 2.2 vocabulary where applicable [9], and `constraint` is an optional ODRL constraint expression. The `since_version` field records which policy version introduced this claim, enabling calling agents to compute diffs between versions. The `category` and `dimension` fields classify the claim by kind of data and kind of operation; these are consumed by extensions (see §6.3) and may be left unspecified in the core.

The key design constraint is that every `PolicyClaim` must have a stable `id` across versions. A claim that changes meaning between versions MUST be assigned a new `id`; the old `id` is retired. This allows calling agents to detect precisely which claims changed when a version bumps.

3.2 ConsentRecord

A `ConsentRecord` documents the calling agent's parsed understanding of and decision about a specific `PolicyDocument` version. Records form a singly-linked list (the *consent chain*) for a given caller-callee pair.

Formally, a `ConsentRecord` R is a tuple:

```
R = (id, prev_id, caller, callee, policy_version, policy_hash,
     parsed_claims, decision, timestamp, valid_until, signature,
     caller_capability_hash, reconsent_trigger)
```

where `prev_id` references the immediately preceding record in the chain (null for the first record), `parsed_claims` contains one `ParsedClaim` entry for every `PolicyClaim` in the referenced `PolicyDocument`, `decision` is one of {accepted, rejected, conditional}, and `valid_until` is either an ISO 8601 timestamp or a sentinel value (`"on_version_bump"`, `"on_capability_change"`, or `"on_any_change"`; see §3.6).

A `ParsedClaim` entry records, for each policy claim, whether the calling agent understood the claim, whether it disputes the claim, and if disputed, a natural-language explanation of the dispute.

Critical invariant: every `PolicyClaim` in the `PolicyDocument` MUST have a corresponding `ParsedClaim` in the `ConsentRecord`. Agents cannot silently ignore inconvenient clauses. This single requirement is what separates the model from a boolean acceptance flag.

Conditional consent is supported such that when `decision` is `conditional`, the calling agent has accepted *some* claims but disputes others. The callee MUST enforce claim-level gating: a skill whose governing `PolicyClaim` is marked `disputed: true` in the `ParsedClaim` array is blocked, while skills governed by undisputed claims remain invocable. This allows the agent to continue operating at reduced capability rather than halting entirely, which is a practical necessity

for long-running agent workflows where a blanket rejection would cascade into downstream failures.

The callee enforces claim-level gating by cross-referencing the `ParsedClaim` array in the caller's `ConsentRecord` against its own skill registry. Each skill in the callee's A2A `AgentCard` (or MCP tool manifest) is annotated with the `claim_id` values that govern it via the `policy_claims` array (see §4.1). When the callee receives a task request, it resolves the caller's active `ConsentRecord`, iterates the requested skill's `policy_claims`, and checks whether any referenced claim is marked `disputed: true` or `understood: false`. If so, the callee MUST reject the invocation with a structured error indicating which claims block the skill.

The linked-list structure of the consent chain is central to the legal value of the model. A chain of records shows the full history of what a calling agent agreed to on behalf of its principal, at the time of each action, rather than just at signup. We used the same linked structure in a prior production human-authentication system to preserve all accepted versions of terms per user for legal auditability. Here it is applied to the agent-to-agent context and extended to include the per-action adherence records described in §3.3.

3.3 AdherenceEvent

An `AdherenceEvent` records the calling agent's runtime evaluation of a policy claim for a specific action attempt. Events form a second singly-linked list (the *adherence trail*) anchored to a `ConsentRecord`.

Formally, an `AdherenceEvent` E is a tuple:

```
E = (id, prev_id, consent_record_id, action, claim_id,
clause_ref,
     decision, reasoning, timestamp, context, signature)
```

where `decision` is one of {permit, deny, escalate} and `reasoning` is a natural-language string explaining why the agent reached that decision.

The `reasoning` field is the mechanism that makes adherence auditable. An agent that records:

> *"Action 'aggregate_sessions' maps to odrl:aggregate on pii:session_data. Policy v2.1.0 §3.4 prohibits this where purpose = behavioural_profiling. Denying."*

has produced something qualitatively different from a system that simply refuses a request. The reasoning is citable, inspectable, and attributable. Auditors can trace any enforcement decision back to the exact clause that governed it.

3.4 Chain Properties

The two linked lists (the consent chain and the adherence trail) together satisfy five properties that are relevant to legal and regulatory accountability.

1. **Completeness.** Every action attempt produces an adherence event. A deny without an event is indistinguishable from a silent failure.
2. **Traceability.** Every adherence event references a consent record, which in turn references a policy document version. The full chain from action to clause is traversable.
3. **Tamper evidence.** Records and events carry optional JWS signatures [10] over their canonical JSON with the `signature` field set to the empty string prior to signing, using the same bootstrapping convention as `PolicyDocument.hash` in §3.1. The resulting signature is then inserted into the record. A verifier can confirm that no record was modified after signing.
4. **Version fidelity.** The `policy_hash` field in the consent record binds the acceptance to the exact document content, independent of URI availability. The hash survives link rot.
5. **Ledger anchoring (optional).** JWS signatures prove *who signed* but not *when*, and they cannot prevent collusion. If both parties agree to rewrite the chain, signatures alone cannot detect it. For deployments requiring third-party tamper evidence, each record may carry a `ChainAnchor`: a cryptographic hash anchored to an external append-only ledger. The model is ledger-agnostic; the anchor may point to a public blockchain (Ethereum, Polygon), a permissioned ledger (Hyperledger Fabric), or a transparency log (RFC 9162). Anchoring is strictly additive and deployments that do not require it simply omit the `chain_anchor` field.

3.5 Formal Verification of the Consent Lifecycle

We specify the consent lifecycle as a TLA+ state machine and verify its safety and liveness properties under model checking. The specification is included in the reference repository (`specification/ConsentLifecycle.tla`) with the corresponding TLC configuration (`specification/ConsentLifecycle.cfg`).

The lifecycle has seven states for a given caller–callee pair: Idle, PolicyFetched, GovernanceReview, Accepted, Rejected, Conditional, and Stale, shown in Figure 1. Table 2 lists the safety and liveness properties verified by TLC.

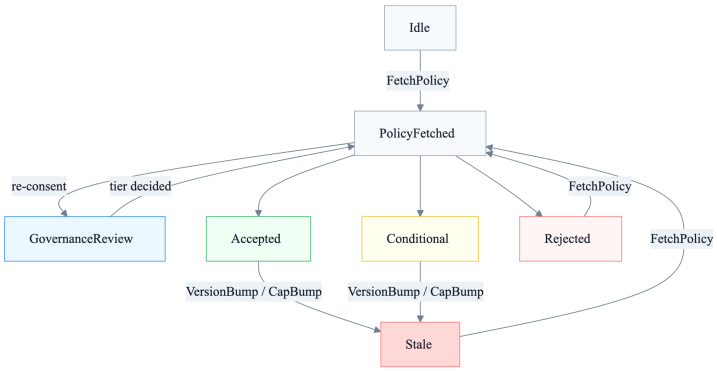

Figure 1: ACAP consent lifecycle state machine. The seven states govern the bind/re-bind cycle between a single caller and callee pair. GovernanceReview is reachable only when the governance-tiering extension is loaded (see §6.3); with the core alone, re-consents flow directly from PolicyFetched to the decision states.

Table 2. Safety and liveness properties verified by TLC.

ID	Property	Kind	Statement
S1	NoSkillWithoutConsent	Safety	A skill call never occurs unless at least one ConsentRecord in the chain has decision $\in$ {accepted, conditional}.
S2	ChainMonotonicity	Safety	`consentChain` and `adherenceTrail` are append-only; lengths never decrease.
S3	AdherenceAnchored	Safety	Every AdherenceEvent references a valid ConsentRecord index.
S4	SkillRequiresPermit	Safety	A skill call is always preceded by at least one AdherenceEvent with decision = permit on an undisputed claim.
S5	ConditionalGating	Safety	A disputed claim always produces deny or escalate, never permit.
S6	NoDisputedPermit	Safety	No adherence event for a disputed claim carries a permit decision.

ID	Property	Kind	Statement
S7	NoSkillOnCapabilityDrift	Safety	No skill call occurs when the caller's capability fingerprint has changed since the ConsentRecord was created.
L1	EventualReConsent	Liveness	Under weak fairness, any staleness (version bump or capability change) eventually leads to a new ConsentRecord.
L2	EventualCapReConsent	Liveness	A capability bump eventually leads to re-consent. The agent does not operate indefinitely under stale reasoning.

Properties S1–S4 establish the core consent-before-action guarantee. S5–S6 formalise conditional gating for reduced-permission operation. S7 addresses agent ephemerality such that capability drift blocks all skill invocation until re-consent is obtained. We note that `caller_capability_hash` is self-reported by the caller and the callee cannot independently verify it; this is a known limitation which we discuss in §6.1. L1 and L2 ensure the protocol does not deadlock after either a policy version bump or a capability change.

The model was checked with TLC using `MaxVersions = 3`, `MaxAdherenceEvents = 4`, and `MaxCapVersions = 2`, covering three policy version bumps, two capability changes, and four adherence events per consent epoch. All nine properties hold with zero violations across the reachable state space. TLC performs bounded model checking such that it exhausts the state space within the declared constant bounds but does not constitute a proof for arbitrary parameter values. The bounds were chosen to cover realistic deployment scenarios; unbounded verification would require inductive proof techniques beyond the scope of the present work.

3.6 Capability-Bound Consent

Human users persist. They create accounts, accumulate consent history, and the consent chain tracks a stable identity across years. Agents are different in kind such that they are spawned, updated, and terminated. An agent instance may exist for minutes or hours, and the next instance, even if it shares the same `caller_agent_id`, may run a different model, carry different tools, or operate under a different context window.

This creates a problem that human consent systems never face: the reasoning entity that consented may not be the reasoning entity that acts. An agent running GPT-4o that recorded `understood: true` for a prohibition on data

aggregation may be replaced by an instance running a fine-tuned variant that interprets "aggregation" differently. The `ParsedClaim` entries in the original `ConsentRecord` are no longer trustworthy, not because they were falsified, but because the entity that produced them no longer exists.

The model addresses this with `caller_capability_hash`, a SHA-256 fingerprint of the agent's model identifier, tool manifest, and reasoning configuration (system prompt, temperature, and any chain-of-thought or retrieval-augmented generation settings that influence how the agent interprets policy claims), recorded on every `ConsentRecord`. The hash is computed over the canonical JSON serialisation (RFC 8785) of a `CapabilityManifest` object containing these fields in alphabetical key order, using the same circularity-breaking convention as `PolicyDocument.hash` in §3.1, such that the `caller_capability_hash` field is set to the empty string before hashing. Three re-consent triggers are defined:

Trigger	Cause	Effect
`POLICY_BUMP`	Callee publishes new PolicyDocument	Re-fetch, diff, re-consent
`CAPABILITY_CHANGE`	Caller's capability hash changes	Re-evaluate all ParsedClaims
`PRINCIPAL_CHANGE`	Human principal identity changes	Full re-consent with new principal_id

The `valid_until` field supports sentinel values that control invalidation granularity: `"on_version_bump"` for callee-side changes only, `"on_capability_change"` for caller-side changes only, or `"on_any_change"` for either side. This lets callees choose their risk tolerance such that a financial-services agent may require `"on_any_change"` while a low-stakes utility agent may accept `"on_version_bump"`.

The consent chain is stored by the callee. It is the callee's audit trail and the callee's legal protection. The chain is keyed by `caller_agent_id`, which is a DID or HTTPS URL bound to the principal or organisation rather than to the process, so new agent instances inherit the chain if they share the identity. Both parties SHOULD maintain independent copies of the chain. The callee's copy is authoritative; the caller's copy provides independent evidence if the callee's ledger is unavailable, tampered with, or disputed. The chain itself is append-only; agent termination does not delete records.

---

4. Protocol Instantiations

An important design constraint of the present work is that the instantiations described in this section do not require changes to the A2A or MCP core specifications. Both protocols already define a native extension mechanism

(`capabilities.extensions` in A2A and the `capabilities` object in MCP), and ACAP is implemented entirely through those mechanisms and through middleware at the caller and callee boundaries. The reference implementation described in §4.3 confirms that a working end-to-end deployment is buildable today with existing A2A and MCP libraries, which means adoption is a matter of installing a middleware library at each participating agent rather than advancing a specification revision.

4.1 Agent2Agent (A2A) Protocol

A2A is an open, vendor-neutral protocol for agent-to-agent communication [4]. Each agent's AgentCard is a JSON document published at `/.well-known/agent-card.json` that advertises the agent's capabilities, skills, and authentication requirements.

A2A supports protocol extensions via the `capabilities.extensions` array [4]. Each extension is declared with a `uri`, a `description`, and a `required` flag. This is the mechanism used by the Agent Payments Protocol (AP2) to advertise payment capability [11], and we use the same mechanism for ACAP.

Step 1: Declare the extension in the AgentCard

```
{
  "capabilities": {
    "extensions": [{
      "uri":
      "https://github.com/ravikiran438/agent-consent-protocol/v0.1"
      ,
      "description": "ACAP v0.1: versioned usage policy and
      consent auditing.",
      "required": true,
      "params": { "minVersion": "0.1", "maxVersion": "0.1" }
    }]
  },
  "usage_policy": {
    "version": "2.1.0",
    "document_uri":
    "https://callee.example.com/.well-known/usage-policy.json",
    "document_hash": "sha256:a3f5c2...",
    "effective_date": "2026-02-27T00:00:00Z",
    "acceptance_required": true,
    "acceptance_endpoint":
    "https://callee.example.com/acap/consent",
    "natural_language_uri": "https://callee.example.com/terms"
  }
}
```

The extension URI is versioned. When a future ACAP version introduces

breaking changes (for example, structured reasoning in a later version), the URI changes accordingly. A calling agent that encounters an unrecognised ACAP version SHOULD fall back to the highest mutually supported version or decline the handshake.

Step 2: Annotate skills with governing claims

Each skill in the AgentCard carries a `policy_claims` array listing the claim IDs that govern its use, mirroring the MCP `policyClaims` annotation in §4.2. This allows the calling agent to evaluate only the relevant claims before invoking each skill:

```
{
  "skills": [{
    "name": "analyse_dataset",
    "policy_claims": ["claim-data-retention",
    "claim-aggregation-prohibition"]
  }]
}
```

Without this mapping, the callee has no mechanism to enforce claim-level gating for conditional consent (§3.2), as it would not know which skills to block when specific claims are disputed.

Step 3: Consent handshake before first skill call

The calling agent fetches the `PolicyDocument`, verifies its hash, parses every `PolicyClaim`, and POSTs a `ConsentRecord` to the `acceptance_endpoint` before invoking any skill. The callee verifies that the record covers every claim and that the `policy_hash` matches the current document. Only then does it permit skill invocation.

Step 4: Per-action adherence recording

Before each skill call, the calling agent evaluates the relevant claims and POSTs an `AdherenceEvent` to the callee. The callee appends the event to the adherence trail for that consent record. The complete flow is shown in Figure 2.

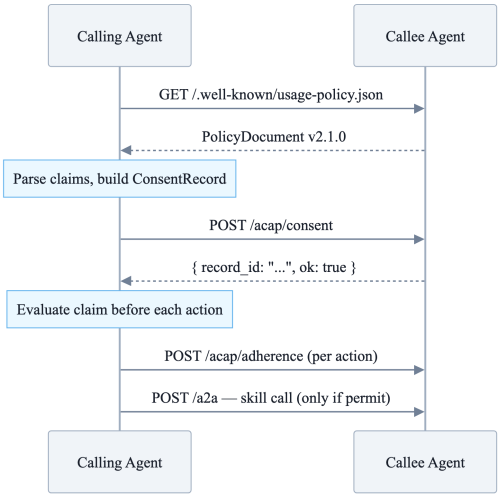

Figure 2: ACAP consent and adherence sequence.

The `POST /acap/adherence` endpoint operates in one of two modes, declared in the AgentCard's `usage_policy` object. In `local` mode the adherence event is fire-and-forget such that the caller records its own decision and the callee appends it to the audit trail. In `delegated` mode the callee evaluates the event and returns an enforcement decision; the caller MUST NOT invoke the skill until it receives a `permit` response. Delegated mode adds a round trip but gives the callee veto authority, which is appropriate for regulated interactions where the callee bears compliance obligations.

When the callee publishes a new `PolicyDocument`, the calling agent detects the version change on the next AgentCard fetch by comparing `usage_policy.version` against the cached consent record's `policy_version`. It then fetches the new document, identifies changed claims via `since_version`, and creates a new `ConsentRecord` with `prev_id` pointing to the now-invalidated record. The old record is never deleted. The current detection mechanism is poll-based (AgentCard fetch). For long-running workflows where polling latency is unacceptable, callees SHOULD additionally support a push channel (webhook callback or SSE endpoint) registered during the consent handshake.

4.2 Model Context Protocol (MCP)

MCP governs communication between an orchestrator agent (client) and tools or data sources (servers) [5]. The MCP specification as of November 2025 [6] does not define a consent primitive analogous to what we propose, and the instantiation below extends MCP using its standard `capabilities` object.

The model maps to MCP as follows. The MCP `initialize` handshake response

includes a `capabilities` object. We add a `usagePolicy` capability:

```
{
  "capabilities": {
    "tools": {},
    "usagePolicy": {
      "version": "1.0.0",
      "documentUri":
      "https://server.example.com/.well-known/usage-policy.json",
      "documentHash": "sha256:b7f3a1...",
      "acceptanceRequired": true
    }
  }
}
```

Individual tool definitions can carry a `policyClaims` annotation listing the claim IDs that govern their use:

```
{
  "name": "aggregate_user_data",
  "description": "Aggregates user data across sessions.",
  "policyClaims": ["claim-pii-aggregate-prohibition"],
  "inputSchema": { ... }
}
```

This allows calling agents to evaluate only the relevant subset of claims before invoking each tool rather than re-evaluating the entire policy document per call. The consent and adherence records use the same schema as the A2A instantiation.

The MCP instantiation is lighter because MCP's trust boundary is typically within a single organisation (orchestrator to owned tools). Cross-organisational A2A calls carry higher legal weight and warrant the fuller handshake described in §4.1.

4.3 Reference Implementation and Overhead

The accompanying repository provides a Python reference implementation of the three structural operations that any ACAP implementation must perform on every interaction: canonical hashing of the `PolicyDocument` per §3.1, structural validation of the consent chain per §3.2, and structural validation of the adherence trail per §3.3–§3.4. The implementation is accompanied by 35 unit tests that exercise the invariants described in those sections.

To characterise the per-call cost of the protocol on commodity hardware, we ran a micro-benchmark of each operation on a 2024 MacBook Air (Apple M4, Python 3.13.5). Each measurement is the median of 200 to 500 samples; the 99th percentile is reported alongside. The policy used for the chain-validation rows contains ten claims, which is a typical size for a production usage policy.

Operation	Median	p99
`compute_policy_hash` (10 claims)	19 μs	28 μs
`compute_policy_hash` (50 claims)	74 μs	88 μs
`compute_policy_hash` (200 claims)	278 μs	324 μs
`validate_consent_chain` (chain length 1)	20 μs	27 μs
`validate_consent_chain` (chain length 5)	96 μs	124 μs
`validate_consent_chain` (chain length 20)	373 μs	458 μs
`validate_adherence_trail` (trail length 10)	3 μs	9 μs
`validate_adherence_trail` (trail length 100)	27 μs	32 μs
`validate_adherence_trail` (trail length 1000)	265 μs	331 μs

These numbers are in microseconds. A typical remote skill invocation between two agents over the public internet sits in the 20–100 ms range such that the ACAP overhead measured here represents well under one percent of the skill-call latency it accompanies, even at chain and trail lengths that are conservative overestimates of steady-state operation.

Two caveats apply. First, the benchmark measures only the in-process structural operations and does not include the network round-trip for `POST /acap/consent` or `POST /acap/adherence`; the actual wall-clock cost of the consent handshake on the wire will be dominated by TCP and TLS setup rather than by the validators themselves. Second, JWS signature verification is not yet implemented in the reference validators and is therefore excluded from the numbers above. We expect signature verification to add a further tens of microseconds per record on the same hardware; this is a natural next step for the reference implementation.

4.4 Demo Deployment

To confirm that the middleware framing in §4 actually composes with existing A2A infrastructure, we built a two-agent end-to-end deployment. The callee is a data-analysis agent implemented as a FastAPI service that publishes an A2A AgentCard at `/.well-known/agent-card.json` and mounts the ACAP callee middleware described in `acap.middleware.callee` at the `/acap` URL prefix. The caller is a marketing-insights agent implemented as an async Python client wrapped in the `ACAPCaller` described in `acap.middleware.caller`. Both agents use Gemini 2.5 Flash as their reasoning backend such that the callee's skill delegates its one-shot dataset analysis to Gemini and the caller uses Gemini to parse each `PolicyClaim` into a `ParsedClaim` during the handshake.

The demonstration scenario is a policy with three prohibitions: a retention limit on session data, a prohibition against aggregation for behavioural profiling, and a prohibition against third-party distribution. The caller declares its intent as producing a marketing insights report that does not engage in behavioural profiling of individual customers. Gemini parses each of the three claims as understood and undisputed under that intent such that the caller arrives at a `ConsentRecord` with `decision = accepted`. The caller then attempts two skill calls in sequence:

the first with `purpose = behavioural_profiling`, which matches the prohibition's constraint and is denied at the adherence layer before the skill is ever invoked, and the second with `purpose = statistical_analysis`, which the prohibition permits and which returns a qualitative summary.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

(.venv) →  agent-consent-protocol git:(main) ✗ python -m demo.caller_agent

────────────────────────────────────────────────────────────────────────────────
1. Consent handshake
────────────────────────────────────────────────────────────────────────────────
ConsentRecord id      : ccde69c3-8edb-41a6-8e5d-7a2960a859df
Decision              : accepted
  - claim-retention                 understood=True  disputed=False
  - claim-aggregation-prohibition   understood=True  disputed=False
  - claim-no-third-party            understood=True  disputed=False

────────────────────────────────────────────────────────────────────────────────
2. First skill call: purpose = behavioural_profiling (expect deny)
────────────────────────────────────────────────────────────────────────────────
AdherenceEvent id     : 65bcd435-f641-4e2b-8d22-817bd2789234
Decision              : deny
Reasoning             : Action 'odrl:aggregate' matches prohibition §3.4 (odrl:aggregate on pii:session_data) under the current c
ontext {'purpose': 'behavioural_profiling'}. Denying.
Caller will NOT invoke the skill — as expected, the adherence layer blocked the call.

────────────────────────────────────────────────────────────────────────────────
3. Second skill call: purpose = statistical_analysis (expect permit)
────────────────────────────────────────────────────────────────────────────────
AdherenceEvent id     : 0f2c5b34-9565-4da1-91d3-a1db6d683884
Decision              : permit
Reasoning             : Action 'odrl:aggregate' is governed by prohibition §3.4 but the prohibition's constraint does not apply u
nder context {'purpose': 'statistical_analysis'}. Permitting.

Skill result:
  summary      : Aggregate statistics for this dataset would provide insights into the typical engagement patterns, summarizing c
entral tendencies and the spread of user session durations and page views. This would allow for an understanding of average user
interaction time and content consumption, as well as the variability in these behaviors across the customer base.
  authorised_by: 0f2c5b34-9565-4da1-91d3-a1db6d683884
```

Figure 3: Caller agent trace: consent handshake (top), blocked skill call on a disputed purpose (middle), and permitted skill call returning a qualitative summary (bottom). Each parsed claim is produced by a Gemini call, and each adherence decision records a natural-language reasoning string as described in §3.3.

Figure 3 shows the caller's trace across the consent handshake and both skill calls. The handshake block displays the three parsed claims as evaluated by Gemini; the deny block shows the reasoning string recorded by the `AdherenceEvent`; and the permit block shows the callee's skill response alongside the `event_id` that authorised it.

```
(.venv) →  agent-consent-protocol git:(main) ✗ uvicorn demo.callee_agent:app --port 8001
INFO:     Started server process [84767]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO:     127.0.0.1:61325 - "GET /.well-known/agent-card.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:61325 - "GET /.well-known/usage-policy.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:61398 - "POST /acap/consent HTTP/1.1" 200 OK
INFO:     127.0.0.1:61398 - "POST /acap/adherence HTTP/1.1" 200 OK
INFO:     127.0.0.1:61398 - "POST /acap/adherence HTTP/1.1" 200 OK
INFO:     127.0.0.1:61398 - "POST /skills/analyse_dataset HTTP/1.1" 200 OK
INFO:     127.0.0.1:61398 - "GET /acap/audit/did%3Aagent%3Amarketing-insights HTTP/1.1" 200 OK
```

Figure 4: Callee server log for the same end-to-end session. Every interaction is a standard HTTP request against a FastAPI app, with the ACAP endpoints (`/acap/consent`, `/acap/adherence`, `/acap/audit`) sitting alongside the A2A-standard well-known paths. No modification of the A2A runtime is required.

Figure 4 shows the callee's HTTP log over the same session, which is

the cleanest evidence that ACAP requires no protocol-level modification: every interaction is a standard HTTP request, and the ACAP-specific endpoints sit alongside the A2A-standard `/.well-known/agent-card.json` and `/.well-known/usage-policy.json` paths under one FastAPI app.

```
────────────────────────────────────────────────────────────────────────────────
4. Audit report fetched from the callee
────────────────────────────────────────────────────────────────────────────────
{
  "consent": {
    "record_id": "ccde69c3-8edb-41a6-8e5d-7a2960a859df",
    "prev_record_id": null,
    "caller_agent_id": "did:agent:marketing-insights",
    "callee_agent_id": "did:agent:data-analysis-agent",
    "policy_version": "1.0.0",
    "policy_hash": "sha256:a80c4dd2dfba3c014978b3e159c6872e70427c70a8ae4708289f2bddda6e9c23",
    "parsed_claims": [
      {
        "claim_id": "claim-retention",
        "understood": true,
        "disputed": false,
        "dispute_reason": null
      },
      {
        "claim_id": "claim-aggregation-prohibition",
        "understood": true,
        "disputed": false,
        "dispute_reason": null
      },
      {
        "claim_id": "claim-no-third-party",
        "understood": true,
        "disputed": false,
        "dispute_reason": null
      }
    ],
    "decision": "accepted",
    "accepted_at": "2026-04-16T09:12:28Z",
    "valid_until": "on_version_bump",
    "caller_signature": null,
    "principal_id": "principal:ravi@example.com"
  },
  "trail": [
    {
      "event_id": "65bcd435-f641-4e2b-8d22-817bd2789234",
      "prev_event_id": null,
      "consent_record_id": "ccde69c3-8edb-41a6-8e5d-7a2960a859df",
      "action": "odrl:aggregate",
      "clause_evaluated": "\u00a73.4",
      "claim_id": "claim-aggregation-prohibition",
      "decision": "deny",
      "reasoning": "Action 'odrl:aggregate' matches prohibition \u00a73.4 (odrl:aggregate on pii:session_data) under the current
context {'purpose': 'behavioural_profiling'}. Denying.",
      "timestamp": "2026-04-16T09:12:28Z",
      "context": {
        "purpose": "behavioural_profiling"
      },
      "agent_signature": null
    },
    {
      "event_id": "0f2c5b34-9565-4da1-91d3-a1db6d683884",
      "prev_event_id": "65bcd435-f641-4e2b-8d22-817bd2789234",
      "c
...
```

Figure 5: Audit endpoint output. The linked-list structure of the consent chain and adherence trail is visible through the `prev_record_id` and `prev_event_id` fields, and the per-action reasoning is preserved verbatim for later inspection.

Figure 5 shows the audit endpoint's response. The linked-list structure of the consent chain and the adherence trail is visible through the `prev_record_id` and `prev_event_id` fields, and the per-action reasoning is preserved verbatim for inspection by a human principal, a compliance officer, or a regulator.

The full deployment code is in the `demo/` directory of the accompanying repository. A reviewer can reproduce Figures 3 through 5 by following the README there; the end-to-end run completes in under ten seconds on the same MacBook Air M4 that produced the benchmark numbers in §4.3. The wall-clock time is dominated by the Gemini inference calls (three during the handshake, one for the permitted skill invocation) rather than by the ACAP validators themselves, which contribute the microsecond-scale overhead reported in §4.3.

The Gemini-backed claim parser used in the demo is illustrative, not norma-

tive. Because the parser is a non-deterministic language model, two runs of the handshake against the same `PolicyDocument` can in principle produce two different `ParsedClaim` arrays such that the resulting `ConsentRecord` values are not byte-identical and therefore not reproducible across runs. A production deployment should instead use a deterministic parser, for example a rule-based ODRL evaluator, a cached LLM output keyed on `(claim_id, caller_intent)` hash, or a constrained-decoding pipeline with a fixed random seed. The caller middleware is designed to accept any implementation of the `ClaimParser` protocol such that an operator can substitute a deterministic parser without changes to any other component. The Gemini-backed parser remains useful during agent development for exploration and for qualitative evaluation of a candidate policy.

---

5. Related Work

A fuller treatment of related work complements the summary in §1.3. We survey proposals by the layer each addresses.

Single-agent governance. OAGS [12] defines five governance primitives (deterministic identity, declarative policy, runtime enforcement, structured audit evidence, cryptographic verification) but is explicitly "local-first." It governs what one agent may do in its own environment, not what two agents agree to when they interact. OpenMandate [15] takes a similar single-agent stance, separating policy from execution using declarative YAML mandates. Both address the right abstraction level but stop at the organisational boundary. Policy Cards [16] make an individual agent's constraints inspectable through machine-readable governance artefacts. The `PolicyDocument` is the callee's equivalent of a Policy Card, but `ConsentRecord` and `AdherenceEvent` capture the bilateral negotiation and ongoing compliance that Policy Cards do not cover.

Runtime enforcement. PCAS [17] compiles Datalog-derived policies into instrumented agents that are policy-compliant by construction. A deployment could use PCAS for local enforcement and ACAP for cross-agent consent; the two address different layers. MI9 [19] focuses on drift detection, identifying when agents deviate from expected behaviour. MI9 detects non-compliance after the fact; ACAP provides the evidentiary basis (what was agreed, what was checked) that drift detection consumes.

Governance architecture. GaaS [18] proposes an external governance agent supervising other agents at runtime. The AIGA Internet-Draft [14] proposes tiered risk-based governance covering action authorisation and audit logging. AIGA governs *what* an agent does; ACAP governs *under what agreed terms*.

Financial services. FINOS AI Governance Framework v2.0 [13] defines MI-21 (Agent Decision Audit and Explainability) with tiered audit logging up to cryptographic tamper-evidence. MI-21 is conceptually close to the adherence trail but is explicitly silent on consent versioning [13]. A FINOS contribution to

address the versioning gap is in preparation.

Privacy and consent standards. IEEE P7012 [20] defines machine-readable personal privacy terms. P7012 addresses the human-to-service relationship; ACAP addresses the agent-to-agent relationship where human preferences must propagate through delegation chains. W3C DPV v2.2 [21] provides standardised vocabularies for data processing activities. The Kantara Consent Receipt [22] established early principles (consent as receipt, machine-readability, user portability) that inform the `ConsentRecord` design.

OAuth ecosystem. UMA 2.0 [26] enables resource owners to control delegated access through an authorisation server. UMA governs *who may access which resource*; ACAP governs *what a calling agent agreed the callee may do* and whether it honoured those terms at every action. RFC 9396 (Rich Authorization Requests) [25] extends OAuth with structured authorization data beyond binary scopes. RAR lets a client express *what* it wants to do, but is a request-time mechanism with no versioning, no per-action adherence, and no re-consent on policy change.

Audit trails. AuditableLLM [23] contributes hash-chain audit trails for language model interactions. Our chain structure shares the pattern but applies it to consent and policy adherence across agent boundaries.

Foundational analysis. Rida [24] examines what consent means when an AI agent, not a human, is the party clicking "I agree." That analysis directly motivates the present work.

ODRL 2.2 [9] provides the vocabulary we use for `PolicyClaim.action` and `PolicyClaim.asset`. We treat ODRL as a vocabulary convention, not a normative dependency. ODRL is a rights *expression* language, not a consent *lifecycle* protocol. It has no `ConsentRecord`, no `AdherenceEvent`, no versioned chain, and no capability-bound invalidation. ACAP uses ODRL vocabulary for claim semantics and addresses the consent lifecycle layer that ODRL does not cover.

Three contributions distinguish this work from the above: the proof-of-adherence framing that shifts from one-time acceptance to per-action, per-clause adherence events with reasoning trails; the inter-agent consent chain as a versioned linked-list audit structure *between* two agents; and capability-bound consent that ties consent validity to the agent's own configuration.

---

6. Discussion

6.1 Threat Model and Limitations

The `reasoning` field on `AdherenceEvent` is free-form natural language. This is a deliberate choice for v0.1 such that the field is immediately useful to human auditors. It also means reasoning strings are not machine-comparable; two agents may evaluate the same clause differently and produce non-equivalent reasoning

strings with no way to detect the discrepancy. A structured reasoning format would enable automated auditing but would raise the implementation barrier substantially, and we defer this.

The model assumes calling agents act in good faith when populating `parsed_claims` and `reasoning`. A malicious agent could record `understood: true` for a prohibition and then violate it. This is worse than the current state in that it produces false evidence, but it also creates a clearer liability standard. An agent that explicitly acknowledged a prohibition and violated it has acted in demonstrable bad faith, which is legally more tractable than silent non-compliance under a clicked-through ToS. Similarly, property S7 guarantees that a capability change triggers re-consent, but the `caller_capability_hash` is self-reported and the callee has no mechanism to independently verify the caller's capability fingerprint. The honest comparison is not "ACAP vs. perfect enforcement" but "ACAP vs. the current state," which is no consent record at all.

A fuller adversarial threat model (adversarial `caller_intent` declaration that steers an LLM-backed claim parser toward a favourable parse, `ConsentRecord` replay across agent identities, and policy-squatting through an initial permissive version followed by a restrictive version bump) is out of scope for the present work. We note that analogous gaps exist in the human-consent layer that ACAP is replacing, since a human signing a contract can also misstate intent and a service can also substitute terms between acceptance and use. The initial deployment context we anticipate is closed enterprise integration in which both the caller's principal and the callee operate under the same organisational trust boundary; the adversarial extensions belong to a subsequent paper alongside remote-attestation integration and signed replay defenses.

6.2 Open Questions

Three questions the community should resolve:

1. **Self-attestation vs. verification.** `ParsedClaim.understood` is currently self-attested. A verifiable proof that the calling agent processed the claim text would close the S7 gap; zero-knowledge proofs of document processing are theoretically possible but practically expensive.

2. **Grace periods.** `valid_until: "on_version_bump"` blocks all skill invocation immediately on version change. For long-running tasks, an immediate block may be disruptive, and a grace period (for example, complete in-flight tasks, block new initiations) needs specifying.

3. **Capability fingerprint granularity.** The `caller_capability_hash` is defined as a SHA-256 digest of model identifier, tool manifest, and reasoning configuration. In practice, what constitutes a "material" capability change is ambiguous. A minor prompt template update may not affect policy reasoning, while a model version bump almost certainly does. The current

design treats any fingerprint change as a re-consent trigger; a future version may introduce a capability diff to distinguish material from immaterial changes.

6.3 Extensions Under Development

Several extensions to the core model are maintained in the accompanying repository as separate proposals. These include tiered escalation via a governance agent, asymmetric sensitivity preferences across data category and usage dimension, structured regulatory context propagation (for frameworks such as HIPAA, GDPR, PCI-DSS, and the EU AI Act), and a plain-English audit projection layer. Each extension is maintained independently with its own `README.md` and `STATUS.md` and is not part of the core normative specification presented here.

---

7. Conclusion

The consent model we inherited from human authentication systems is a timestamped acceptance record, and it is inadequate for autonomous agent protocols. It proves acceptance. It cannot prove adherence. Agents are capable of something humans are not, namely the ability to parse policy documents, evaluate clauses at runtime, and produce reasoning trails for every action. The three primitives proposed here (`PolicyDocument`, `ConsentRecord`, and `AdherenceEvent`) provide the infrastructure to make that capability useful. The versioned linked-list audit chain gives legal accountability to the calling agent's principal. The A2A and MCP instantiations in §4 demonstrate that ACAP deploys against today's agent protocol infrastructure without requiring any changes to the A2A or MCP core specifications. Adoption is a matter of installing a middleware library at each participating agent, not of advancing a specification revision.

The timing matters. The EU AI Act's high-risk enforcement date of August 2, 2026 is months away. The A2A specification is under active revision by a Linux Foundation working group, and the FINOS AI Governance Framework has an open gap in MI-21 on consent versioning. This is an unusually short window in which a protocol proposal can land in multiple active governance processes simultaneously.

I built a version of this system for humans, years ago at a consumer technology company. Linked consent chains, immutable versions, the whole audit trail. Users spent four seconds on the acceptance screen and the chain proved acceptance with precision but proved nothing else. Agents do not need four seconds. They need a protocol that asks them to actually read the document, evaluate each clause, and leave a record of what they decided and why. That is what ACAP provides.

References

[1] AI Agent Index 2025. arXiv:2602.17753, February 2026. (The report covers the 2025 landscape; the preprint was published in February 2026.)

[2] EU AI Act. Regulation (EU) 2024/1689. https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai

[3] EU AI Office. AI Act Service Desk: Frequently Asked Questions. https://ai-act-service-desk.ec.europa.eu/en/faq (As of March 2026, no FAQ or guidance document addresses consent mechanisms for autonomous agent-to-agent transactions.)

[4] A2A Protocol Specification. https://a2a-protocol.org/latest/specification/

[5] Model Context Protocol Specification. https://modelcontextprotocol.io/specification/

[6] MCP November 2025 Changelog. https://modelcontextprotocol.io/specification/2025-11-25/changelog

[7] Uniform Electronic Transactions Act (UETA), §14. https://www.uniformlaws.org/

[8] Proskauer: Contract Law in the Age of Agentic AI, 2025. https://www.proskauer.com/blog/contract-law-in-the-age-of-agentic-ai-whos-really-clicking-accept

[9] W3C ODRL Information Model 2.2. https://www.w3.org/TR/odrl-model/

[10] RFC 7515: JSON Web Signature (JWS). https://datatracker.ietf.org/doc/html/rfc7515

[11] AP2: Agent Payments Protocol. https://github.com/google-agentic-commerce/ap2

[12] Ngozo, J.F. Open Agent Governance Specification (OAGS). Sekuire, 2026. https://sekuire.ai/blog/introducing-open-agent-governance-specification

[13] FINOS AI Governance Framework v2.0. https://air-governance-framework.finos.org/

[14] Aylward, J. et al. AIGA: AI Governance and Accountability Protocol. IETF Internet-Draft, draft-aylward-aiga-1. https://datatracker.ietf.org/doc/draft-aylward-aiga-1/

[15] McDonough, R. OpenMandate: Governing AI Agents by Authority, Not Instruction. Law://WhatsNext, 2026. https://lawwhatsnext.substack.com/p/openmandate-governing-ai-agents-by

[16] Mavračić, J. Policy Cards: Machine-Readable Runtime Governance for Autonomous AI Agents. arXiv:2510.24383, 2025.

[17] Palumbo, N. et al. PCAS: Policy Compiler for Secure Agentic Systems. arXiv:2602.16708, 2026.

[18] Gaurav, S. et al. Governance-as-a-Service: A Multi-Agent Framework for AI System Compliance and Policy Enforcement. arXiv:2508.18765, 2025.

[19] Wang, C.L. et al. MI9: An Integrated Runtime Governance Framework for Agentic AI. arXiv:2508.03858, 2025.

[20] IEEE P7012: Standard for Machine-Readable Personal Privacy Terms. https://standards.ieee.org/ieee/7012/

[21] W3C Data Privacy Vocabulary (DPV) v2.2. https://w3c.github.io/dpv/dpv/

[22] Kantara Initiative: Consent Receipt Specification v1.1. https://kantarainitiative.org/

[23] Li, D., Yu, G., Wang, X. and Liang, B. AuditableLLM: A Hash-Chain-Backed, Compliance-Aware Auditable Framework for Large Language Models. Electronics, 15(1), 56. MDPI, 2025.

[24] Rida, C. When an AI Agent Says 'I Agree,' Who's Consenting? TechPolicy.Press, December 2025. https://www.techpolicy.press/when-an-ai-agent-says-i-agree-whos-consenting/

[25] RFC 9396: OAuth 2.0 Rich Authorization Requests. https://datatracker.ietf.org/doc/html/rfc9396

[26] Kantara Initiative. *User-Managed Access (UMA) 2.0 Grant for OAuth 2.0 Authorization*. January 2017. https://kantarainitiative.org/uma-specifications/